

Bonus Assignment - Survey

Serhan Yilmaz - srhnylmz14@gmail.com

Columbia CS PhD Applicant, Fall 2025

Section 1: Implementation Result

1. Please record the command you use to create, run, and evaluate the framework.

- Create:

```
python create.py --config ./serhan_yilmaz.json --output-dir  
./framework_teacher
```

- Run:

```
python model_api.py --input-dir ./framework_teacher
```

- Evaluation:

```
python eval.py --model_api http://127.0.0.1:8000/eval/chat --config  
./serhan_yilmaz.json --documents_dir ./framework_teacher --output-dir  
./framework_teacher
```

Evaluation Results:

Looking at the metrics from goal_completion.json, the implementation worked pretty well:

```
{  
  "user_task_completion": 1.0,  
  "user_task_completion_efficiency": 4.0,  
  "bot_goal_completion": 1.0  
}
```

The numbers tell a clear story - the agent did what it was supposed to do. Perfect task completion (1.0) means it didn't drop the ball on any tasks. The efficiency score of 4.0 shows it got things done in about 4 turns on average, which is pretty good. And the goal completion rate of 1.0 means it stayed on track with its objectives.

Sample Conversations:

Looking through labeled_data.json and simulate_data.json, here are some key conversation patterns that show what the agent can do:

1. Technical Explanations:

User: "How does the RAGWorker use FAISS for info retrieval?"

Assistant: "The RAGWorker uses FAISS (Facebook AI Similarity Search) as part of its retrieval process... [detailed technical explanation follows]"

The agent knows its stuff and can break down complex topics without getting too academic about it.

2. Setup Help:

User: "How do I set up a booking service bot using the framework?"

Assistant: "Setting up a booking service bot involves several steps... [step-by-step guide follows]"

Good at walking users through setup processes step by step.

3. Integration Support:

User: "How do I integrate it with a database?"

Assistant: "Integrating your booking service bot with a database involves... [detailed technical instructions follow]"

Handles technical questions well and keeps track of what the user's trying to do.

4. Framework Concepts:

User: "How is user goal completion efficiency calculated?"

Assistant: "User goal completion efficiency is calculated by determining the average number of turns... [detailed explanation follows]"

Can explain framework concepts in plain language that makes sense.

What stands out from these conversations is that the agent: - Keeps track of what you're talking about across multiple messages - Gives accurate technical info without getting too theoretical - Adjusts its explanations based on what you're asking - Provides practical solutions you can actually use

2. Please copy and paste a sample conversation where you interacted with your created Agent.

Hello! Let's discuss custom agents and the Agent First Organization framework. Can you share your current understanding and specific needs regarding custom agents?

You: if I have configured a simple customer service bot, how can I customize its workers?

getAPIBotResponse Time: 22.06426978111267

Bot: To customize the workers of your customer service bot, you'll need to focus on defining and potentially creating new worker classes that enhance or modify the bot's behavior. Here's a step-by-step guide to help you customize the workers:

1. **Understand Existing Workers:**

- Review the existing workers defined in your bot's configuration. Each worker has a specific role and set of tasks it performs. Understanding their current functionalities will help you decide what customizations are needed.

2. **Identify Customization Needs:**

- Determine what additional tasks or improvements you want your bot to handle. For instance, you might want it to integrate with a new data source,

provide more detailed responses, or perform specific actions based on user interactions.

3. ****Create Custom Worker Classes:****

- If your customization requires new functionalities, consider creating custom worker classes. These classes should extend the existing worker architecture to implement the desired tasks.
- Define the logic and methods necessary for the new functionalities in your custom worker class.

4. ****Modify the Configuration:****

- Update your bot's configuration file to include the new or modified workers. Make sure to specify the correct worker class names and any additional parameters they might require.

Example snippet for a custom worker in JSON configuration:

```
```json
"workers": [
 {
 "name": "CustomResponseWorker",
 "type": "CustomWorkerClass",
 "tasks": [
 {
 "task_name": "FetchAdditionalInfo",
 "actions": [
 {
 "action_type": "HTTPRequest",
 "method": "GET",
 "url": "https://api.example.com/info"
 }
]
 }
]
 }
]
```
```

5. ****Integrate with Taskgraph:****

- Ensure your new workers are appropriately integrated into the taskgraph. Define how tasks flow between workers and what conditions trigger each task.

6. ****Test the Customized Bot:****

- Thoroughly test your bot to ensure that the new workers perform their tasks as expected. Check for any issues or unexpected behaviors and refine your customizations as needed.

7. ****Deploy and Monitor:****

- Once testing is complete, deploy your updated bot into a production environment. Monitor its performance and gather user feedback to make further improvements.

By following these steps, you can effectively customize the workers of your customer service bot to meet specific requirements. If you need more detailed guidance on any part of this process, feel free to ask!

3. **Which of the following components did I use to create my Agent? (*Check all that apply.*)**

- ☐ Interactive Panel to customize task planning -> experimented with it but did not use it in the final implementation
- ☒ Taskgraph Generation
- ☐ Worker Customization
- ☒ Evaluation

Section 2: Feedback on Components

4. **For the fields in the config file, Which field(s) in the config file were most useful for creating my Agent? Which field(s) did I find least important? Are there additional fields or features I would suggest adding to improve the Agent's capabilities?**

The most useful fields were the `task_docs` and `rag_docs` as they formed the knowledge base for my teaching assistant. These fields were crucial in providing accurate and contextual responses about the framework. The `role`, `user_objective`, and `builder_objective` fields were also vital in shaping the assistant's behavior and ensuring it maintained its teaching focus.

The least important field was the `domain` field, as the teaching assistant's domain was inherently clear from its role and objectives.

Suggested additional fields: - Response style configuration for different user expertise levels - Error handling preferences - Integration points for external knowledge bases - Performance metric thresholds

5. **Were the instructions for constructing the config file clear? Please specify areas where improvements are needed.**

The instructions were generally clear, particularly regarding the basic structure and required fields. However, I encountered some ambiguity regarding the `num` parameter in `task_docs` and `rag_docs`. Initially, I set higher numbers, this led to unnecessary crawling attempts. Clearer documentation about URL handling and crawling behavior would be beneficial.

6. **How well did the generated task planning (without modification) cover the tasks I expected the Agent to handle?**

- ☒ Most
7. **How useful were the steps generated in the task planning (without modification)? Rate their usefulness on a scale from 1 to 5. (1: Not Useful at All, 5: Very Useful)**

- ☒ 3
8. **If I used the Interactive Panel for customizing task planning, how useful was it? Rate its usefulness on a scale from 1 to 5. (1: Not Useful at All, 5: Very Useful)**

- ☐ 1

- ☒ 2

- ☐ 3

- ☐ 4

- ☐ 5

Note: I did not use the Interactive Panel in my implementation. All configuration was done through the configuration file directly.

9. **Follow up to question 8: Based on my experience in customizing task planning: How much of the original task planning did I modify?**

- ☐ 80%+

- ☐ 60%+

- ☐ 40%+

- ☐ 20%+

- ☒ <20%

Note: Since I did not use the Interactive Panel, this question is not applicable to my implementation.

10. **If I designed my own worker, what are they and how were they used?**

I did not design or customize any workers. Instead, I utilized the standard workers provided by the framework in their default configuration: - RAGWorker: For retrieving information from my documentation sources - RagMsgWorker: For handling message-

based interactions - MessageWorker: For managing conversation flow - SearchWorker: For documentation search capabilities - DefaultWorker: For handling fallback scenarios

The power of my implementation came from properly configuring these standard workers through the `serhan_yilmaz.json` file, particularly in the careful selection and organization of documentation sources in the `task_docs` and `rag_docs` sections.

11. **Follow up to question 10: How difficult did I find customizing the Worker framework? Rate the difficulty on a scale from 1 to 5. (1: Very Easy, 5: Very Difficult)**

- ☒ 3 -> As I said, I did not customize the workers as it seemed rather complicated, and I already had good results at that point.

12. **How useful was the user simulator for evaluating my Agent? Rate its usefulness on a scale from 1 to 5. (1: Not Useful at All, 5: Very Useful)**

- ☒ 5

13. **What issues did I identify in my Agent during evaluation, and how did I address them?**

There were some issues that I had to address:

1. **Selenium WebDriver Configuration:** The system initially failed due to incorrect Chrome WebDriver path specifications. I resolved this by properly configuring the WebDriver path in system environment variables and ensuring Chrome version compatibility.
2. **Platform Compatibility:** The `requirements.txt` file required modification for Windows compatibility. I removed packages that were only necessary for Mac/Linux environments, ensuring smooth operation on Windows systems.
3. **Documentation Crawling:** Initial configuration attempted to crawl multiple pages from single-page documentation sources. I corrected this by setting all `num` values to 1 in both `task_docs` and `rag_docs` configurations.
4. **Port Configuration:** The port configuration in the model API is a prime example - the default port assignment is hardcoded without any fallback mechanism, which caused problems with my router configuration. I had to manually modify the code to use port 8000. The `build_rag.py` script also needed significant work - its document processing pipeline isn't robust enough for real-world use. The script assumes ideal conditions and fails ungracefully when dealing with complex documentation structures or when storage operations don't go as planned.

Section 3: Feedback on Framework

14. **How would I rate the overall difficulty of using the Agent First Organization framework? (1: Very Easy, 5: Very Difficult)**

- ☒ 3

15. What improvements would I suggest for the framework?

Based on my implementation work, I found several areas that need improvement. The documentation, while extensive, needs significant work. It covers basic concepts but lacks practical examples and clear explanations of critical components. The URL crawling documentation was particularly problematic - the `num` parameter's behavior isn't properly explained, which led to a lot of wasted time trying different configurations that didn't work as expected.

The URL crawler is fundamentally flawed in its current state. I extensively tested different values for the `num` parameter, hoping to get automated crawling working, but the crawler consistently failed to handle documentation sites properly. I ended up having to manually add all documentation URLs to my config file - a tedious workaround that defeats the purpose of having a crawler. The framework needs a proper web crawler that can intelligently traverse documentation structures, following links and building a comprehensive knowledge base from a single entry point.

The technical implementation has several rough edges that need smoothing out. The port configuration in the model API is a prime example - the default port assignment is hardcoded without any fallback mechanism, which caused problems with my router configuration. I had to manually modify the code to use port 8000. The `build_rag.py` script also needed significant work - its document processing pipeline isn't robust enough for real-world use. The script assumes ideal conditions and fails ungracefully when dealing with complex documentation structures or when storage operations don't go as planned.

The framework's biggest limitation is its approach to real-world deployment. Most commercial websites now implement sophisticated anti-bot measures, making it practically impossible to build customer service bots that need to access operator documentation. I tried building test cases for telecom operator documentation (AT&T, Vodafone) and hit immediate roadblocks due to anti-bot protections. This isn't just an edge case - it's a fundamental limitation that makes the framework unsuitable for many real-world applications. The framework needs to either implement proper API integration capabilities or develop alternative methods for documentation ingestion that can work with modern websites.

The debugging experience is particularly frustrating. When things go wrong with the RAG pipeline or port configuration, the error messages are cryptic and unhelpful. I spent hours debugging issues that could have been solved in minutes with better error reporting. The framework needs comprehensive logging and debugging tools that can help developers quickly identify and fix issues in their implementations.

16. Additional comments or suggestions?

For my implementation, I chose an interesting direction by creating a teaching assistant for the framework itself. This meta-application helped me explore the framework's capabilities while also revealing some areas for potential enhancement.

I'm quite pleased with the framework's core functionality. My evaluation metrics demonstrated excellent task completion rates and conversation efficiency, which speaks to the soundness of the underlying architecture. The framework performs particularly well in controlled environments where documentation access is straightforward, though there are opportunities to enhance its adaptability to various real-world scenarios.

I am curious to see similar systems built on top of this framework, which are self-aware and can adapt to the user's needs, all while being able to self-improve themselves. Enabling information access for agents on a wider basis paves a really interesting direction of research, as recently demonstrated by OpenAI's Deep Research enabled on the O3 models. Gives me chills just thinking about it, that would be awesome!